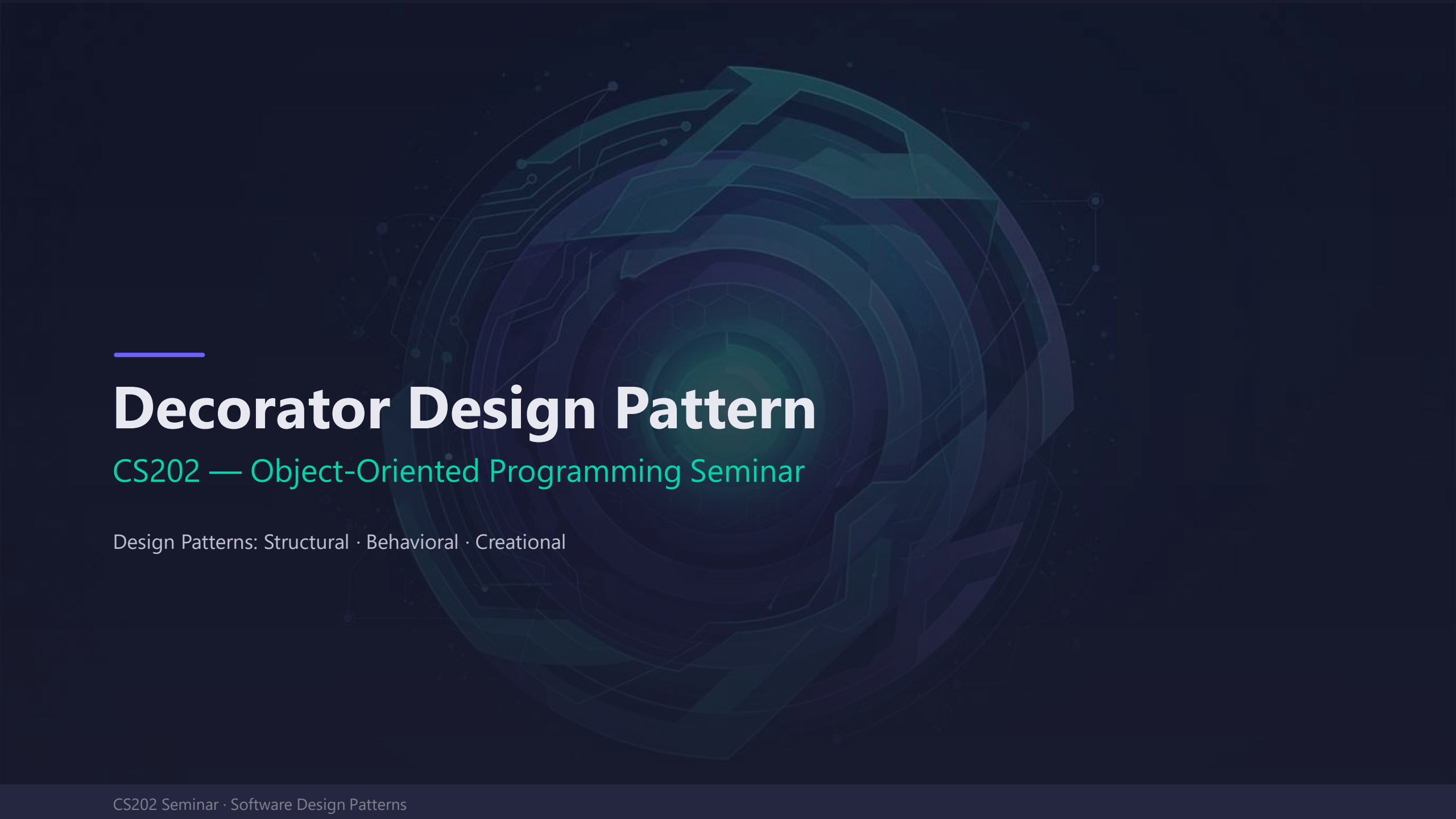




Decorator Design Pattern

CS202 — Object-Oriented Programming Seminar

Design Patterns: Structural · Behavioral · Creational

Agenda

01 Real-world Problem

02 Naive Solutions

03 Problems with Naive Approaches

04 Decorator Pattern Introduction

05 UML Class Diagram

06 C++ Implementation

07 Execution Flow & Output

08 Advantages & Disadvantages

09 Strategy Pattern

10 Facade Pattern

11 Real-world Applications

12 Mini Quiz

Real-world Problem

Coffee Shop Order System

You own a coffee shop. Your system needs to handle beverages with various condiments:

- Dark Roast, House Blend, Espresso, Decaf
- Milk, Mocha, Soy, Whipped Cream

Each combination changes the description and the total cost of the order.

The Challenge:

How do you design a flexible system that supports unlimited condiment combinations without modifying existing code?



Naive Solution 1: Inheritance

Create a subclass for every possible combination:



⚠ Class Explosion Problem

With 4 beverages × 4 condiments (each optional):

$4 \times 2^4 = 64$ subclasses needed!

Each new condiment doubles the number of classes.

Naive Solution 2: Boolean Flags

```
class Beverage {  
    // Boolean flags for each condiment  
    bool milk, mocha, soy, whip;  
  
    double cost() {  
        double total = baseCost();  
  
        if(milk) total += 5;  
        if(mocha) total += 10;  
        if(soy) total += 8;  
        if(whip) total += 7;  
  
        return total;  
    }  
};
```

Looks simpler, but...

X Adding a condiment = modify the class

Violates Open-Closed Principle

X Can't add two shots of the same condiment

Boolean = on or off, no quantity

X cost() becomes a maintenance nightmare

Every new condiment adds another if-block

X Not all condiments apply to all beverages

Tea with whip? Espresso with soy milk?

Problems with Naive Approaches



Class Explosion

Every combination of beverage + condiment requires a separate subclass.

$N \text{ beverages} \times 2^k \text{ condiments} = \text{too many classes}$



Hard Maintenance

Changing the price of a condiment requires editing dozens of classes.

Bug-prone, time-consuming updates



OCP Violation

Open-Closed Principle: classes should be open for extension, closed for modification.

Both approaches require modifying existing code



No Runtime Flexibility

Can't add or remove condiments at runtime — everything is fixed at compile time.

Users want dynamic customization

Decorator Pattern Introduction

The Solution: Dynamic Object Wrapping

Instead of using inheritance to add behavior at compile time, use **Composition** to wrap objects dynamically at runtime.

🔑 Key Insight

The wrapper (Decorator) implements the same interface as the object it wraps!

- Attach additional responsibilities to an object dynamically.
- Flexible alternative to subclassing for extending functionality.
- Wrappers can be stacked in any sequence or combination.



Formal Definition (GoF)

"Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality."

— Design Patterns (Gang of Four, 1994)

1. Wrapper Concept

Also known as **Wrapper**.
Encloses a target object
inside a decorator object.

Passes requests to target
and performs additional
actions before/after.

2. Interface Matching

Decorator conforms to
the component interface.

Client treats wrapped
objects identically to
unwrapped base objects.

Transparent to client!

3. Recursive Nesting

Decorators can wrap
other Decorators.

Creates an arbitrary
chain/pipeline of
behaviors executed in
nested order.

Core Design Principle

Open-Closed Principle (OCP)

"Classes should be OPEN for extension, but CLOSED for modification."

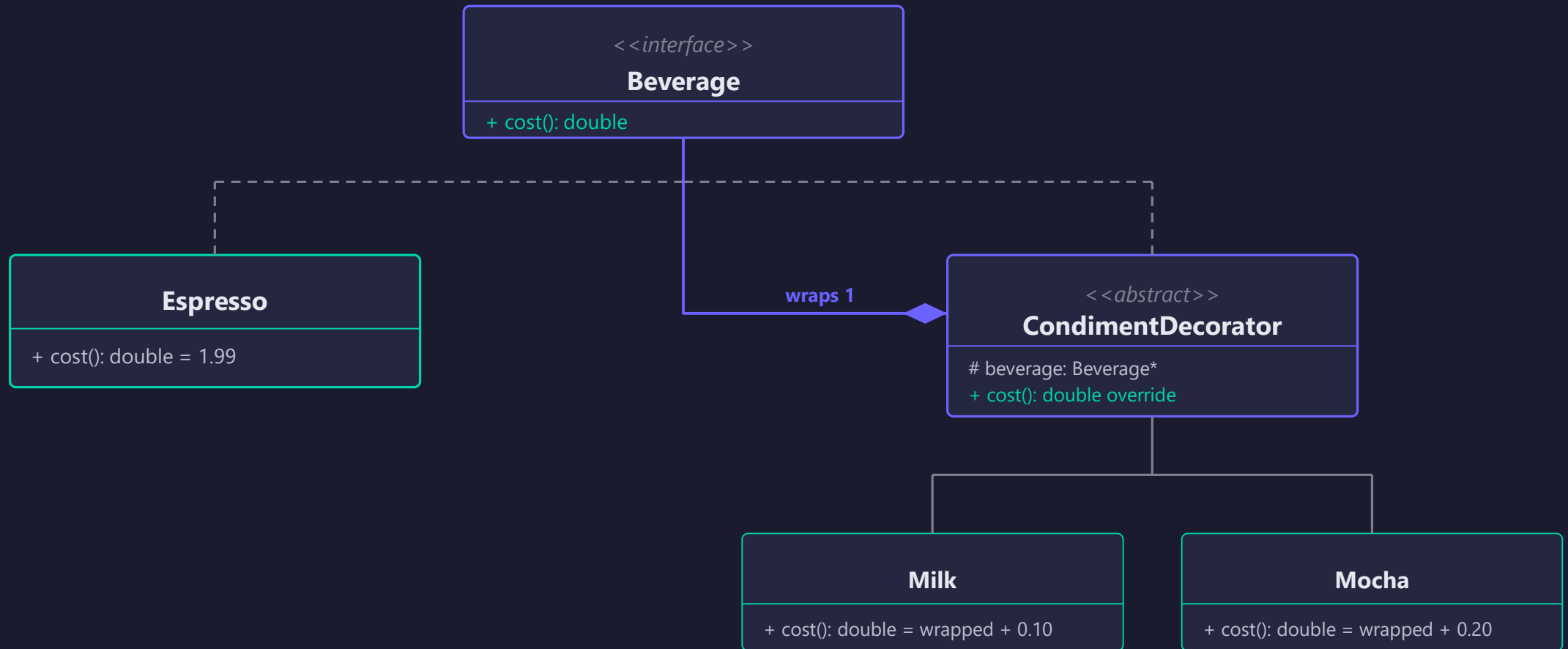
Open for Extension

- You can add new behavior or features at any time.
- Simply create a new Concrete Decorator subclass.
- Dynamically compose new behaviors without altering existing code.

Closed for Modification

- Existing tested code remains untouched.
- Base components don't know about decorators wrapping them.
- Eliminates risk of introducing bugs into working production code.

UML Class Diagram



Class Roles in Decorator Pattern

1. Component (Base Interface)

- Defines the interface for objects that can have responsibilities added to them dynamically.
- Abstract base class or pure interface.

Example: Beverage

2. ConcreteComponent

- Defines an object to which additional responsibilities can be attached.
- Represents core basic functionality.

Example: Espresso, DarkRoast

3. Decorator (Abstract Wrapper)

- Maintains a reference to a Component object and defines an interface that conforms to Component's interface.

Example: CondimentDecorator

4. ConcreteDecorator

- Adds responsibilities to the component.
- Overrides methods and calls wrapped object's methods + adds extra state/behavior.

Example: Milk, Mocha, Whip

C++ Implementation: Component Interface

Beverage.h

```
#pragma once
#include <string>

class Beverage {
protected:
    std::string description = "Unknown Beverage" ;

public:
    virtual ~Beverage() = default ;

    virtual std::string getDescription () const { return description; }

    virtual double cost () const = 0 ; // Pure virtual method

};
```

C++ Implementation: Concrete Components

Espresso.h

```
class Espresso : public Beverage {
public:
    Espresso () {
        description = "Espresso";
    }

    double cost () const override {
        return 1.99 ;
    }
};
```

DarkRoast.h

```
class DarkRoast : public Beverage {
public:
    DarkRoast () {
        description = "Dark Roast";
    }

    double cost () const override {
        return 0.99 ;
    }
};
```

C++ Implementation: Abstract Decorator

CondimentDecorator.h

```
#include "Beverage.h"

class CondimentDecorator : public Beverage {
protected:
    Beverage* beverage ; // Wrapped object pointer

public:
    CondimentDecorator (Beverage* b) : beverage(b) {}

    virtual ~CondimentDecorator() {
        delete beverage; // Clean up wrapped object
    }

    // All concrete decorators must override getDescription() and cost()
};
```

C++ Implementation: Concrete Decorators

Milk.h

```
class Milk : public CondimentDecorator {
public:
    Milk (Beverage* b)
        : CondimentDecorator(b) {}

    std::string getDescription ()
        const override {
        return beverage->getDescription()
            + ", Milk" ;
        }

    double cost () const override {
        return beverage->cost() + 0.10 ;
    }
};
```

Mocha.h

```
class Mocha : public CondimentDecorator {
public:
    Mocha (Beverage* b)
        : CondimentDecorator(b) {}

    std::string getDescription ()
        const override {
        return beverage->getDescription()
            + ", Mocha" ;
        }

    double cost () const override {
        return beverage->cost() + 0.20 ;
    }
};
```

C++ Client Code Assembly

main.cpp

```
int main () {  
  
    // 1. Order a plain Dark Roast ($0.99)  
    Beverage* b1 = new DarkRoast ();  
    std::cout << b1->getDescription() << " $" << b1->cost() << std::endl;  
  
    // 2. Order Dark Roast with Double Mocha + Milk ($0.99 + 0.20 + 0.20 + 0.10)  
    Beverage* b2 = new DarkRoast ();  
  
    b2 = new Mocha (b2);    // Wrap with 1st Mocha  
    b2 = new Mocha (b2);    // Wrap with 2nd Mocha!  
    b2 = new Milk (b2);     // Wrap with Milk  
  
    std::cout << b2->getDescription() << " $" << b2->cost() << std::endl;  
    delete b1; delete b2; return 0 ;  
}
```


Object Wrapping Sequence

Step 1: Core Object



DarkRoast

Base component created.

getDescription():

"Dark Roast"

cost():

\$0.99

Step 2: Wrap with Mocha



Mocha Decorator

DarkRoast

Wrapped with Mocha (+ \$0.20)

getDescription():

"Dark Roast, Mocha"

cost():

\$1.19

Step 3: Wrap with Milk



Milk Decorator

Mocha Decorator (wraps DarkRoast)

Wrapped with Milk (+ \$0.10)

getDescription():

"Dark Roast, Mocha, Milk"

cost():

\$1.29

Execution Flow & Delegated Calls

How `cost()` propagates through wrappers:

1. Client calls `whip->cost()`

Whip adds \$0.15 to whatever `mocha->cost()` returns.

2. Whip delegates to `mocha->cost()`

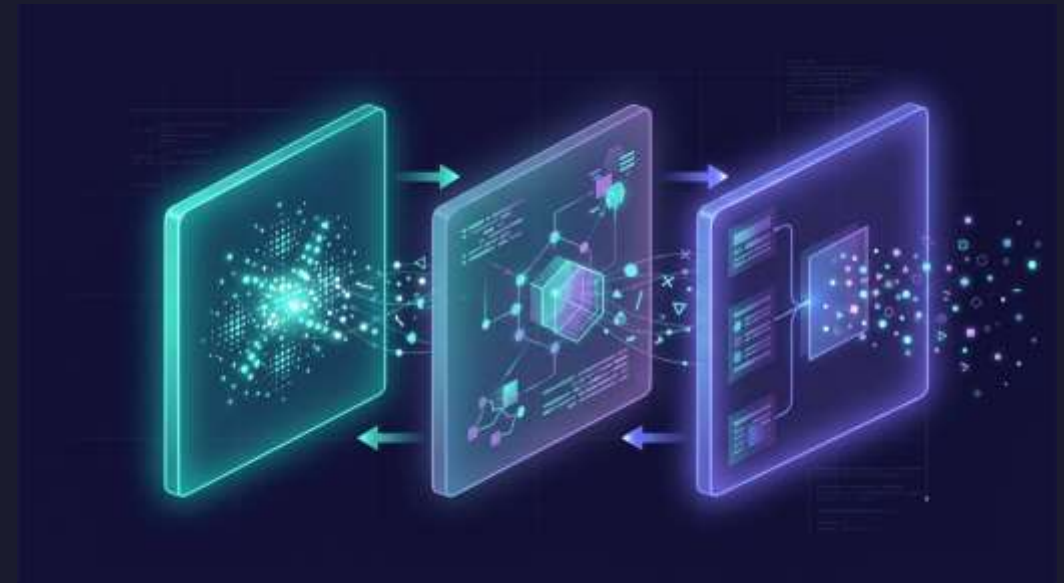
Mocha adds \$0.20 to whatever `darkRoast->cost()` returns.

3. Mocha delegates to `darkRoast->cost()`

DarkRoast (ConcreteComponent) returns base cost: \$0.99.

4. Return Stack: \$0.99 → \$1.19 → \$1.34

Total cost calculated dynamically: \$1.34!



Advantages of Decorator Pattern



Greater Flexibility than Static Inheritance

Responsibilities can be added and removed at runtime simply by attaching and detaching decorators.



Avoids Feature-Laden Classes High Up the Hierarchy

Pay-as-you-go approach: define simple classes and add functionality incrementally with decorators.



Adheres strictly to the Open-Closed Principle (OCP)

New features are added via new decorator classes without modifying existing components.



Supports Mix-and-Match Combinations

Mix decorators in any order or repeat decorators (e.g., Double Whip, Double Mocha) effortlessly.

Disadvantages of Decorator Pattern



Lots of Small Objects

A design using Decorator often results in systems with many small, similar-looking objects.
Can be confusing for developers who are unfamiliar with the codebase.



Complex Initialization Code

Instantiating a heavily decorated component requires nested constructor calls.
Mitigated by using Factory or Builder patterns alongside Decorator.



Component Identity Issues

A decorator acts as a transparent enclosure, but it is not identical to the component itself.
Code that relies on specific concrete types (rtti/dynamic_cast) will break when decorated.

When to Use Decorator Pattern

✓ Dynamic Behavior Addition

When you want to add responsibilities to individual objects dynamically and transparently, without affecting other objects.

Example: Adding scrolling or borders to a UI window

✓ Withdrawal of Responsibilities

When responsibilities can be withdrawn dynamically as needed by unwrapping the object chain.

Example: Enabling/disabling encryption on a data stream

✓ Subclassing is Impractical

When extension by subclassing is impractical due to an explosion of independent extensions producing a massive class hierarchy.

Example: GUI widget options (scrollbar, shadow, border)

✓ Class Definition is Hidden / Sealed

When a class definition is hidden or otherwise unavailable for subclassing (e.g. final class or third-party binary library).

Example: Wrapping a 3rd party stream class

Related Pattern: Strategy Pattern

Interchangeable Algorithms

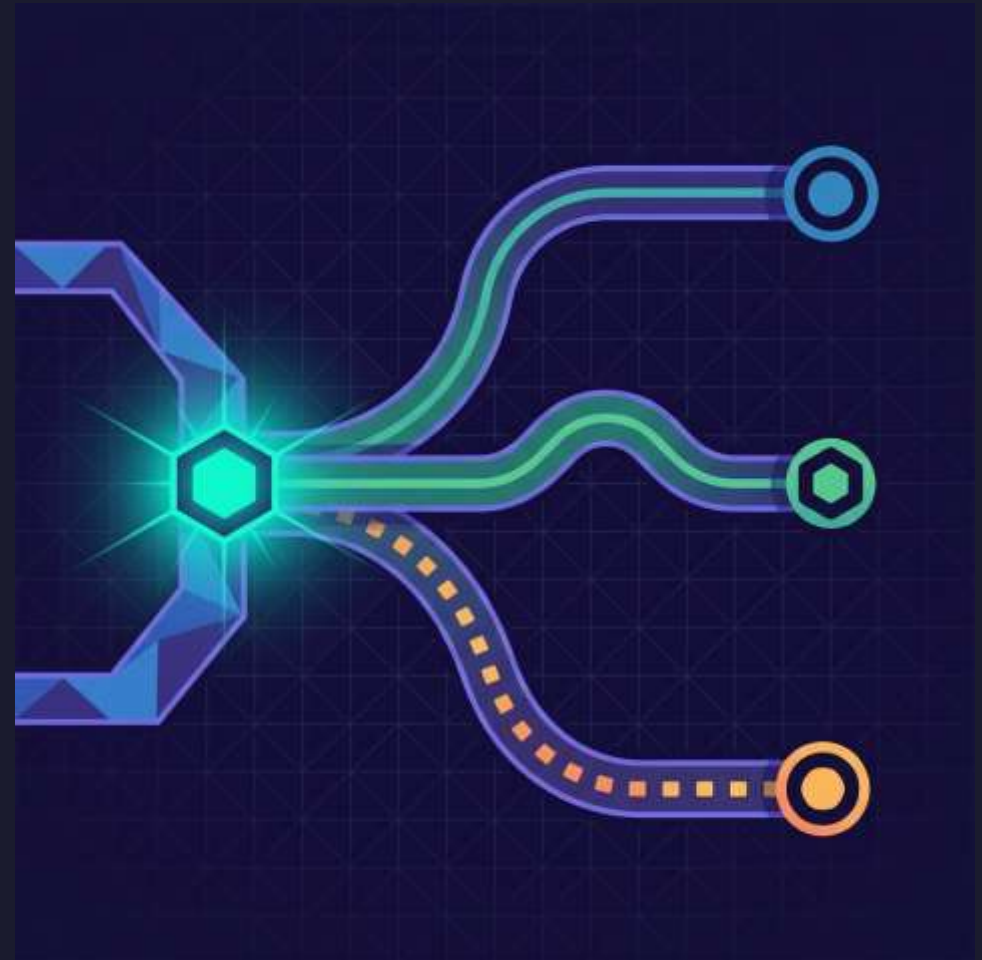
The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them **interchangeable**.

💡 Core Difference in Intent

Strategy changes the **GUTS** of an object.

Decorator changes the **SKIN** of an object.

- Strategy lets the algorithm vary independently from clients.
- Objects hold a reference to a Strategy object and delegate.
- Example: Payment strategies (Credit Card, PayPal, Crypto).



Strategy Pattern Structure & Code

StrategyExample.cpp

```
// 1. Strategy Interface
class PaymentStrategy {
public:
    virtual void pay (int amount) = 0 ;
};

// 2. Concrete Strategies
class CreditCardPay : public PaymentStrategy { ... };

// 3. Context Object
class ShoppingCart {
private:
    PaymentStrategy* strategy;
public:
    void setPaymentStrategy (PaymentStrategy* s ) { strategy = s; }
    void checkout (int amt) { strategy->pay(amt); }
};
```

Decorator vs. Strategy

Feature	Decorator Pattern	Strategy Pattern
Intent	Adds responsibilities to an object dynamically ("changes the skin")	Encapsulates interchangeable algorithms ("changes the guts")
Interface	Decorator implements the exact same interface as the component	Strategy interface is distinct from the Context object
Composition	Recursive nesting: Decorator can wrap another Decorator infinitely	Single delegation: Context holds one Strategy instance
Awareness	Component is completely unaware of being decorated	Context knows about Strategy and explicitly invokes it

Strategy Pattern Summary

Key Takeaways:

1. Encapsulate what varies:

Isolate different algorithms into separate strategy classes.

2. Favor Composition over Inheritance:

Context delegates behavior to a strategy object rather than inheriting implementation.

3. Runtime Flexibility:

Strategies can be swapped dynamically while the program is running (e.g. changing sorting algorithm).

4. Synergies with Decorator:

A strategy can be wrapped inside a decorator to combine interchangeable logic with layered behavior!

Related Pattern: Facade Pattern

Simplified Interface to Subsystems

Provides a unified, simplified interface to a complex set of interfaces in a **subsystem**.

💡 Core Difference in Intent

Facade **SIMPLIFIES** the interface.

Decorator **EXTENDS** the interface without changing it.

- Defines a higher-level interface that makes subsystem easier to use.
- Hides complex internal inter-module dependencies from client.
- Example: One-button `HomeTheater.watchMovie()` API.



Facade Pattern Structure & Code

HomeTheaterFacade.cpp

```
// Subsystem components (Amplifier, Tuner, DvdPlayer, Projector, Lights)
class HomeTheaterFacade {
private:
    Amplifier* amp; DvdPlayer* dvd; Projector* proj; TheaterLights* lights;

public:
    HomeTheaterFacade(Amplifier* a, DvdPlayer* d, Projector* p, TheaterLights* l)
        : amp(a), dvd(d), proj(p), lights(l) {}

    // One simple method hides complex multi-step calls
    void watchMovie (std::string movie) {
        lights->dim(10);
        proj->on(); proj->wideScreenMode();
        amp->on(); amp->setSurroundSound(); amp->setVolume(5);
        dvd->on(); dvd->play(movie);
    }
};
```

Decorator vs. Facade

Decorator Pattern

Primary Purpose:

- Adds or enhances responsibilities of an object.
- Does NOT alter the underlying interface.

Interface Relationship:

- Same interface as wrapped component.
- Client is unaware of decoration (transparent).

Nesting & Recursion:

- Highly recursive — decorators can wrap other decorators in a stack.

Motto: "Same interface, extra behavior"

Facade Pattern

Primary Purpose:

- Simplifies access to an entire subsystem.
- Creates a NEW, simpler interface.

Interface Relationship:

- Brand new, convenient interface.
- Client interacts only with the facade API.

Nesting & Recursion:

- Non-recursive — acts as a single gateway to multiple underlying subsystem objects.

Motto: "New interface, simpler access"

Facade Pattern Summary

Key Takeaways:

1. Principle of Least Knowledge (Law of Demeter):

Talk only to your immediate friends. Facade reduces client dependencies on complex subsystems.

2. Decouples Subsystem from Clients:

Clients don't need to know how subsystem classes work together or manage their lifecycles.

3. Does NOT Prevent Direct Access:

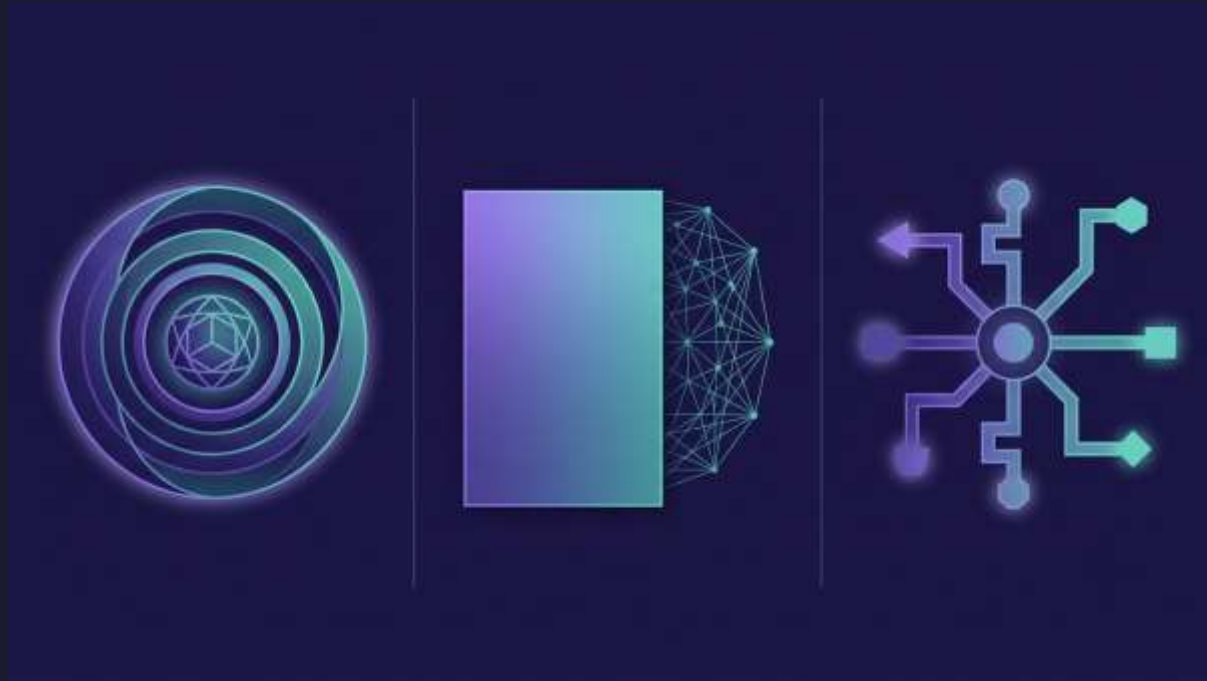
Facade provides a convenient interface for general tasks, but advanced clients can still access subsystem classes directly.

4. Combining Patterns:

A Facade can use Decorators internally to dynamically configure subsystem components before invocation!

Pattern Comparison: Decorator vs Strategy vs Facade

Three structural & behavioral patterns working together to manage complexity:



Decorator

Wraps object to add behavior
Same interface, layered features

Facade

Simplifies entire subsystem
New simplified interface

Strategy

Swaps algorithms dynamically
Interchangeable implementation

Real-world Example: Java I/O Streams

Java's `java.io` package is the textbook implementation of the Decorator Pattern!

JavaIOExample.java

```
// Combining File + Buffered + GZIP input streams
InputStream in =
    new GZIPInputStream (           // Decorator 2: Decompresses data
        new BufferedInputStream (  // Decorator 1: Adds buffering
            new FileInputStream ( "data.txt.gz" ) // Base Component: Reads raw file
        ));
```

Real-world Example: Web Middleware

HTTP Request Processing Pipeline

In modern backend frameworks (Express.js, ASP.NET, Go Gin), middleware functions form a Decorator pipeline:

1. Logging Middleware

Logs request timestamp & IP, then calls next handler.

2. Authentication Middleware

Verifies JWT token. Rejects if invalid, else calls next.

3. Compression Middleware

Compresses response body with Gzip before sending.

4. Core Route Controller

ConcreteComponent that handles business logic and returns data.



Mini Quiz: Test Your Understanding (Part 1/5)

Question 1:

Which GoF pattern would you choose if you need to attach dynamic logging and caching features to an existing Third-Party Stream object without modifying it?

- A) Strategy Pattern
- B) Decorator Pattern
- C) Facade Pattern

Question 2:

What is the main structural difference between Decorator and Facade?

- A) Decorator retains component interface; Facade defines a new simplified interface. ✓
- B) Decorator cannot be nested; Facade can be nested recursively.
- C) Facade alters behavior at compile time; Decorator alters behavior at runtime.

Mini Quiz: Test Your Understanding (Part 1/5)

Question 1:

Which GoF pattern would you choose if you need to attach dynamic logging and caching features to an existing Third-Party Stream object without modifying it?

A) Strategy Pattern

B) Decorator Pattern ✓ (Correct! Wraps object dynamically while sharing interface)

C) Facade Pattern

Question 2:

What is the main structural difference between Decorator and Facade?

A) Decorator retains component interface; Facade defines a new simplified interface. ✓

B) Decorator cannot be nested; Facade can be nested recursively.

C) Facade alters behavior at compile time; Decorator alters behavior at runtime.

Mini Quiz: Interface & Combinatorics (Part 2/5)

Question 3:

Why must the Base Decorator class implement or inherit from the Component interface?

- A) To allow decorators to inherit private state variables from components.
- B) To guarantee interface conformance so clients treat wrapped objects transparently.
- C) To prevent other subclasses from extending the Component interface.

Question 4:

How does Decorator avoid class explosion compared to static inheritance for N features?

- A) Inheritance needs 2^N subclasses, while Decorator composes features dynamically at runtime.
- B) Static inheritance executes faster because decorators require template metaprogramming.
- C) Decorator requires modifying existing source code of the base component.

Mini Quiz: Interface & Combinatorics (Part 2/5)

Question 3:

Why must the Base Decorator class implement or inherit from the Component interface?

- A) To allow decorators to inherit private state variables from components.
- B) To guarantee interface conformance so clients treat wrapped objects transparently. ✓**
- C) To prevent other subclasses from extending the Component interface.

Question 4:

How does Decorator avoid class explosion compared to static inheritance for N features?

- A) Inheritance needs 2^N subclasses, while Decorator composes features dynamically at runtime. ✓**
- B) Static inheritance executes faster because decorators require template metaprogramming.
- C) Decorator requires modifying existing source code of the base component.

Mini Quiz: Principles & Trade-offs (Part 3/5)

Question 5:

Which Object-Oriented Design Principle is most directly promoted by the Decorator Pattern?

- A) Single Responsibility Principle (SRP) only.
- B) Open-Closed Principle (OCP) — open for extension, closed for modification.
- C) Liskov Substitution Principle (LSP) violation.

Question 6:

What is a primary downside or maintenance risk of heavily using Decorators?

- A) It breaks encapsulation by exposing private member variables of the component.
- B) It produces many small nested objects, increasing stack depth and debugging complexity.
- C) Decorators cannot be instantiated dynamically in modern C++.

Mini Quiz: Principles & Trade-offs (Part 3/5)

Question 5:

Which Object-Oriented Design Principle is most directly promoted by the Decorator Pattern?

- A) Single Responsibility Principle (SRP) only.
- B) Open-Closed Principle (OCP) — open for extension, closed for modification. ✓**
- C) Liskov Substitution Principle (LSP) violation.

Question 6:

What is a primary downside or maintenance risk of heavily using Decorators?

- A) It breaks encapsulation by exposing private member variables of the component.
- B) It produces many small nested objects, increasing stack depth and debugging complexity. ✓**
- C) Decorators cannot be instantiated dynamically in modern C++.

Mini Quiz: Implementation & Real-World (Part 4/5)

Question 7:

How do smart pointers (`std::unique_ptr` / `std::shared_ptr`) assist C++ Decorator designs?

- A) They cleanly manage ownership of wrapped component instances, preventing memory leaks.
- B) They automatically convert Decorator into a Strategy pattern at compile time.
- C) They disable virtual table overhead for decorated function calls.

Question 8:

In Java Standard I/O (`java.io`), which code snippet represents a classic Decorator?

- A) `new BufferedReader(new FileReader("file.txt"))` — `BufferedReader` adds buffering capability.
- B) `List<String> list = new ArrayList<>();`
- C) `DatabaseConnection.getInstance();`

Mini Quiz: Implementation & Real-World (Part 4/5)

Question 7:

How do smart pointers (`std::unique_ptr` / `std::shared_ptr`) assist C++ Decorator designs?

- A) They cleanly manage ownership of wrapped component instances, preventing memory leaks. ✓**
- B) They automatically convert Decorator into a Strategy pattern at compile time.
- C) They disable virtual table overhead for decorated function calls.

Question 8:

In Java Standard I/O (`java.io`), which code snippet represents a classic Decorator?

- A) `new BufferedReader(new FileReader("file.txt"))` — `BufferedReader` adds buffering capability. ✓**
- B) `List<String> list = new ArrayList<>();`
- C) `DatabaseConnection.getInstance();`

Mini Quiz: Execution & Identity (Part 5/5)

Question 9:

When wrapping an object with multiple Decorators (e.g. Encryption & Compression), does order matter?

- A) No, decorators are order-independent and execute simultaneously.
- B) Yes, order matters because each layer delegates sequentially (e.g. Encrypt-then-Compress).
- C) Order only matters if decorators use multiple inheritance.

Question 10:

What happens to object identity (e.g. pointer comparison `this == wrapped`) in Decorator?

- A) The Decorator shares the exact same memory address as the inner component.
- B) The Decorator has a distinct identity; code relying on pointer identity (`==`) may fail.
- C) Compilers automatically adjust pointer comparisons for decorated objects.

Mini Quiz: Execution & Identity (Part 5/5)

Question 9:

When wrapping an object with multiple Decorators (e.g. Encryption & Compression), does order matter?

- A) No, decorators are order-independent and execute simultaneously.
- B) Yes, order matters because each layer delegates sequentially (e.g. Encrypt-then-Compress). ✓**
- C) Order only matters if decorators use multiple inheritance.

Question 10:

What happens to object identity (e.g. pointer comparison `this == wrapped`) in Decorator?

- A) The Decorator shares the exact same memory address as the inner component.
- B) The Decorator has a distinct identity; code relying on pointer identity (==) may fail. ✓**
- C) Compilers automatically adjust pointer comparisons for decorated objects.

Summary & Key Takeaways

1 Composition over Inheritance

Decorator uses object composition to attach responsibilities at runtime without class explosion.

2 Transparent Wrapping

Decorators conform to the same interface as wrapped components, enabling transparent nesting.

3 Open-Closed Principle (OCP)

Extend functionality by adding new decorators without editing existing core code.

4 Pattern Synergies

Decorator works seamlessly with Strategy (swapping algorithms) and Facade (simplifying subsystems).

Thank You!

Questions & Discussion

CS202 Seminar · Object-Oriented Software Design Patterns

Decorator Pattern · Strategy Pattern · Facade Pattern